

.NET Self-Learning Questions and Answers

Session 2: .NET Fundamentals

1. Write a comparison between .NET Framework, .NET Core, and .NET 8.

- .NET Framework is the older, Windows-only platform traditionally used for building heavy desktop and web applications tied strictly to Windows environments.
- .NET Core was built from the ground up to be modern, fast, and completely cross-platform, meaning it runs seamlessly on Windows, Mac, and Linux.
- .NET 8 is the latest unified version that brings all these technologies together, offering the best performance, cross-platform capabilities, and long-term support for building any type of modern application.

2. Write the differences between CLR and JVM.

- CLR (Common Language Runtime) is the execution engine for .NET applications, while JVM (Java Virtual Machine) serves the exact same purpose for Java applications.
- CLR is designed to support multiple programming languages (like C#, F#, and VB.NET) by compiling them into a shared intermediate language, whereas the JVM is primarily focused on executing Java bytecode.
- While the JVM acts as a strict virtual barrier to ensure the code runs identically everywhere, the CLR interacts more closely with the underlying operating system to provide performance optimizations.

3. Write Managed Code and Unmanaged Code with suitable examples.

- Managed code is completely controlled by the .NET runtime (CLR), which automatically handles background tasks like memory management, security checks, and garbage collection.
- Unmanaged code runs directly on the operating system outside of the CLR's control, meaning the developer is fully responsible for manually allocating memory and preventing memory leaks.
- An example of managed code is a standard C# web or console application, while an example of unmanaged code is a low-level C or C++ application that uses raw system pointers.

4. Write the purpose of CLS and CTS in .NET.

- CTS (Common Type System) sets the strict rules for how data types are declared and managed in memory, ensuring that an integer in C# is structurally identical to an integer in VB.NET.
- CLS (Common Language Specification) is a specific subset of the CTS rules that a programming language must strictly follow if it wants to seamlessly communicate with

other .NET languages.

- Together, they ensure that different languages can easily share libraries and work together flawlessly within the shared .NET ecosystem without data mismatches.

5. Describe the role of JIT Compiler in the .NET execution lifecycle.

- The Just-In-Time (JIT) compiler translates the intermediate language (IL) code produced by .NET into native machine code that the computer's actual hardware can understand.
- Instead of compiling the entire application at once, it compiles the code exactly at the moment it is needed (on the fly) right before it is executed.
- This dynamic approach helps optimize application performance because the JIT compiler adapts and generates the most efficient native code specifically for the hardware it is currently running on.

Session 4: Static Members & Inheritance

1. Differentiate between Static Constructor and Instance Constructor.

- A static constructor is designed to initialize static data and runs only once automatically before any object is created or any static property is accessed.
- An instance constructor is used to initialize normal class variables and runs multiple times, specifically every single time you create a new object using the 'new' keyword.
- You cannot pass parameters to a static constructor since it runs automatically, whereas instance constructors can be overloaded to accept various different parameters.

2. Write the restrictions of a Static Class.

- A static class is completely restricted from being instantiated, meaning you are never allowed to create objects of it using the 'new' keyword.
- It is strictly limited to containing only static members, static methods, and a static constructor; standard instance properties are not allowed.
- A static class is sealed by default, which means it cannot inherit from any other classes, and no other class is allowed to inherit from it.

3. Write Method Overriding and Method Hiding.

- Method overriding allows a child class to provide a completely new and specific implementation for a virtual method defined in the parent class using the 'override' keyword.
- Method hiding occurs when a child class creates a new method with the exact same name as the parent's method using the 'new' keyword, which intentionally ignores and hides the parent's logic.
- With overriding, the child's method is executed even if the object is accessed through a parent reference, while with hiding, the parent's method will execute if accessed through a parent reference.

4. What are Sealed Methods? Write with examples.

- A sealed method is a method that has already been overridden from a parent class but is

mathematically locked to prevent any further child classes from overriding it again.

- You declare a sealed method by writing the 'sealed' keyword right next to the 'override' keyword in the derived class.
- This feature is highly useful when you want to provide a specific implementation of a feature and strictly guarantee that no other developer can alter its behavior further down the inheritance chain.

5. Write Singleton Pattern and Template Method Pattern.

- The Singleton Pattern is a design structure that restricts a class so that only one single instance of it can ever be created, providing a global point of access to that one object.
- The Template Method Pattern defines the basic skeleton of an algorithm in a parent class but allows child classes to fill in specific details without changing the overall sequence of steps.
- The Singleton focuses entirely on managing object creation efficiently, whereas the Template Method focuses on organizing behavior and maximizing code reuse across different subclasses.

Session 7: Generics & Collections

1. When should List be preferred over Dictionary?

- You should prefer a List when you simply have a collection of items that you want to access sequentially or by their numerical index, like a basic list of names or products.
- A Dictionary is preferred when you need to store your data as paired keys and values, making it incredibly fast to look up specific information using a unique identifier.
- Lists are better suited for sorting and looping over ordered data, while Dictionaries are the best choice for fast, instant data retrieval based on specific IDs or names.

2. Write the use cases of HashSet, Queue, and Stack.

- A HashSet is perfectly used when you need a collection that automatically filters out duplicate items and provides extremely fast mathematical operations like checking for overlapping data.
- A Queue operates on a strict First-In-First-Out (FIFO) rule, making it ideal for real-world scenarios like handling customer service requests in order or managing a printer's task list.
- A Stack operates on a Last-In-First-Out (LIFO) rule, which is the perfect structure for implementing "undo" features in software or navigating backward through a web browser's history.

3. Write the internal working of Dictionary with hashing and collision concepts.

- Internally, a Dictionary uses a standard array to store its data and relies on a mathematical "hash function" to convert your unique key into an exact array index location.
- If two completely different keys accidentally produce the exact same array index, a "collision" occurs, which the Dictionary handles by storing multiple items at that single index using a linked list.

- This highly optimized hashing mechanism is what allows the Dictionary to locate and retrieve values almost instantly, regardless of whether it holds ten items or ten million items.

4. Differentiate between IComparable and IComparer.

- IComparable is implemented directly inside the class itself to define one default sorting rule, such as telling a Student class that it should always be sorted by its ID number.
- IComparer is implemented in completely separate, external classes to create custom sorting logic, allowing you to sort Students by Name in one place and by Grades in another.
- IComparable gives your object a standard, built-in way to sort itself, while IComparer allows for ultimate flexibility to apply many different sorting styles without modifying the original object.

Session 9: Exception Handling & Events

1. Differentiate between throw; and throw ex;

- Using 'throw;' safely pushes the caught error further up the application while preserving the exact error trail (stack trace) and the exact line number where the problem originated.
- Using 'throw ex;' intentionally resets the error trail, making it falsely appear as if the exception started right there in the catch block rather than its true original source.
- It is considered a strict best practice to always use 'throw;' so that developers can easily track down the root cause of bugs without losing valuable debugging history.

2. Write the purpose of checked and unchecked.

- The 'checked' keyword forces the compiler to actively monitor numerical calculations and throw an immediate error if a number becomes too large for its assigned data type to hold.
- The 'unchecked' keyword explicitly tells the compiler to ignore these memory limits, allowing overflowing numbers to silently wrap around into negative values without crashing the software.
- These keywords are crucial when working with heavy financial or scientific calculations where you need strict control over how memory overflows are handled by the system.

3. What are Inner Exceptions?

- An Inner Exception occurs when an application catches one error but decides to wrap it inside a brand-new, more descriptive error before throwing it again.
- The new exception stores the original error details inside its 'InnerException' property so that the true, original root cause of the crash is never permanently lost.
- This approach is extremely helpful for building layered architectures, such as catching an ugly database connection failure and wrapping it in a polite, user-friendly "Service Unavailable" message.

4. Write the Observer Pattern.

- The Observer Pattern is a popular architectural design where a central object (the subject) automatically notifies a list of dependent objects (the observers) the moment its internal state changes.
- It creates a loosely coupled system because the central subject does not need to know the specific details or logic of the observers; it simply broadcasts the update to everyone subscribed.
- A common real-world example of this pattern is a YouTube channel acting as the subject, automatically pushing notifications to all of its subscribed users whenever a new video goes live.

5. Compare Observer Pattern using Interfaces and Events.

- Implementing this pattern manually using Interfaces requires writing extra code to define explicit subscriber contracts and manually managing lists to add or remove listeners.
- Using C# built-in Events highly simplifies this whole process because the C# language automatically manages the subscription lists and background notifications for you.
- Events provide a much cleaner and safer approach, specifically preventing external classes from accidentally clearing the subscriber list or forcing a notification to trigger improperly.

Session 10: LINQ & Extension Methods

1. Write Deferred Execution in LINQ.

- Deferred execution means that when you write a LINQ query, it does not actually run against the database or list immediately; it simply saves the instructions.
- The query is only truly executed at the exact moment you attempt to loop through the results or pull the data into a list, allowing you to modify the query multiple times before running it.
- This massively improves application performance and memory usage because data is fetched and processed only when it is strictly required by the application interface.

2. What are Side Effects in LINQ?

- A side effect in LINQ happens when a query unexpectedly alters external variables, objects, or application states outside the immediate boundaries of the query itself.
- Because LINQ is designed primarily for safely fetching and filtering data in a functional way, modifying data states directly inside a LINQ statement is considered a very poor practice.
- Queries that produce side effects become highly unpredictable and difficult to debug, especially since deferred execution means those modifications might run at unexpected times.

3. Compare LINQ with Java Streams.

- Both LINQ in .NET and Streams in Java exist to provide developers with a highly readable, functional, and declarative way to query and manipulate large collections of data.
- LINQ is deeply and natively integrated into the C# language syntax using SQL-like query

keywords, whereas Java Streams rely strictly on chaining standard methods together.

- LINQ offers incredibly comprehensive support for seamlessly translating queries directly into database commands (like via Entity Framework), whereas Java often relies on external libraries for similar database translation.

4. Write the purpose of `SelectMany()`, `GroupBy()`, and `ToDictionary()`.

- `SelectMany()` is used specifically to flatten complicated, nested lists (like a list of orders containing lists of items) into one single, easily searchable sequence.
- `GroupBy()` takes a massive flat list of data and organizes it neatly into categorized chunks based on a shared property, like organizing a list of employees by their specific department.
- `ToDictionary()` is used to immediately execute a query and structure the final results into key-value pairs, setting up the data for incredibly fast lookups later in the application.

Session 11: Reflection, Attributes & File I/O

1. Write Reflection in .NET.

- Reflection is a highly advanced .NET feature that allows a running program to inspect its own structural code, reading information about hidden classes, methods, and properties at runtime.
- It grants developers the power to dynamically create objects or execute methods purely based on their string names, even if the application did not know about them when it was compiled.
- While it is incredibly powerful for building flexible tools or plugin systems, Reflection is generally slower than normal direct code execution and must be used carefully to avoid performance drops.

2. Compare Reflection in .NET and Java.

- Both .NET and Java offer very robust Reflection APIs that allow applications to dynamically inspect and manipulate their code structures at runtime in very similar ways.
- In .NET, Reflection is tightly paired with custom "Attributes" to read metadata attached to classes, while Java relies heavily on "Annotations" to achieve the exact same structural purpose.
- .NET provides advanced dynamic features like Expression Trees and the 'dynamic' keyword that can make runtime code execution slightly more optimized and flexible compared to Java's traditional approach.

3. Write JSON Serialization.

- JSON Serialization is the crucial process of converting complex C# objects from computer memory into a clean, lightweight, text-based JSON format so it can be saved or sent over the internet.
- Deserialization is the exact reverse process: taking raw JSON text received from an external API or file and securely converting it back into usable C# objects for the

application.

- This process acts as the universal language for modern web development, allowing completely different systems (like a C# backend and a React frontend) to smoothly communicate.

4. Write IDisposable Pattern and using Statement.

- The IDisposable pattern is specifically implemented to manually clean up unmanaged resources like open database connections or file streams that the automatic garbage collector cannot easily close.
- The 'using' statement provides a clean, automated shortcut block in C# that ensures the Dispose method is instantly called once an object is done being used, even if the program crashes.
- Relying on this combination guarantees that your software will not suffer from silent memory leaks or accidentally lock system files by leaving them open in the background.

5. Write the purpose of HttpClient.

- HttpClient is the primary built-in class in .NET used specifically for sending HTTP network requests (like GET or POST) and receiving data from external web services or APIs.
- It is heavily designed to be created once and reused throughout the entire application lifecycle to prevent opening too many network connections and exhausting the server's resources.
- It natively relies on asynchronous programming, which means your application's user interface will not freeze up while it waits for a slow internet API to respond.

Sessions 13-22: ASP.NET Core, EF Core, Web API & React

1. Compare ADO.NET and Entity Framework.

- ADO.NET is a low-level, manual data access tool where developers are required to write raw SQL queries by hand and manually translate database rows into C# objects.
- Entity Framework (EF) is a high-level tool that automatically writes complex SQL queries behind the scenes and magically maps database tables directly to C# classes.
- ADO.NET provides maximum, fine-tuned execution speed and control, while Entity Framework offers massive developer productivity by drastically reducing repetitive database code.

2. Write the advantages of EF Core.

- EF Core allows developers to interact with the database using purely C# code and LINQ, virtually eliminating the need for developers to learn or write complex SQL strings.
- It is extremely lightweight and features full cross-platform support, making it perfectly optimized for modern cloud deployments and microservice architectures.
- It features powerful "Database Migrations," allowing developers to update their actual

database tables simply by altering their C# models and running a single command.

3. Write the architecture of ASP.NET Core Web API.

- ASP.NET Core Web API is built on a highly clean, modular architecture specifically designed to receive incoming HTTP web requests and return raw data (usually JSON) rather than full web pages.
- It deeply integrates built-in Dependency Injection to easily manage background services, making the application extremely easy to test and cleanly separated.
- It utilizes a dynamic pipeline of "Middlewares," allowing developers to easily snap in modular features like security authentication, error handling, and routing as data flows in and out.

4. Write the deployment process of a Web API on IIS.

- Deploying to IIS (Internet Information Services) starts by publishing the .NET project into a specific folder containing the compiled DLL code and web configuration files.
- The server must have the ASP.NET Core Hosting Bundle installed so that IIS knows exactly how to capture web traffic and route it to the .NET runtime engine.
- Finally, you configure an IIS website pointing to the published folder and set the Application Pool to "No Managed Code," since the .NET Core application manages its own internal execution.

5. Write React and ASP.NET Core Integration.

- Modern integration usually follows a cleanly decoupled architecture where ASP.NET Core acts as an independent backend API serving JSON data, and React acts as a completely separate frontend consuming it.
- During the development phase, the React local server is often configured to quietly proxy requests back to the ASP.NET API to bypass Cross-Origin Resource Sharing (CORS) security blockers.
- For the final production phase, the React application is compiled down into plain HTML and JS files, which can either be hosted together inside the ASP.NET application or completely separate on a CDN.